

Relazione homework 1

Petrina Gaia - s308033, Ruggiero Alessandro - s309582, Ruscitto Virginia - s307256

Introduzione:

L'obiettivo del seguente homework è confrontare lo spettro di energia ottenuto tramite il calcolo esplicito della DFT e quello calcolato dalla funzione di libreria della FFT. Verranno utilizzati due file musicali con caratteristiche diverse, in modo tale da poter effettuare due confronti su diversi spettri di frequenza e riuscire anche ad osservare le caratteristiche spettrali di due generi musicali diversi. In questo caso sono stati analizzati il brano rock "*Smells Like Teen Spirit*" dei Nirvana e la composizione classica "*Morning Mood*" di Edvard Grieg.

L'homework è stato svolto usando Python.

Procedimento e codice:

Il main è diviso in 3 parti: il confronto fra la DFT e la FFT del brano classico, il confronto fra la DFT e la FFT del brano rock e il confronto fra le FFT dei due brani. La prima e la seconda parte sono identiche (i nomi della variabili sono uguali, ma seguiti da un 1 per la prima parte e da un 2 per la seconda).

Innanzitutto, il main richiede all'utente di inserire un valore per il parametro M , che può essere sia di tipo *int* che *float*, e che rappresenta la durata delle sotto-finestre in cui verrà diviso il segnale audio. Per garantire un confronto equo, specie nella terza parte, il parametro M rimane lo stesso per tutto l'arco del programma. In seguito, viene invocata la funzione *apri_file*

La funzione *apri_file* apre un file audio, ne verifica la correttezza e lo divide in sotto-finestre di durata M . Per fare ciò riceve in input una stringa con il nome del file audio, il parametro M e le due variabili *inizio* e *durata*, che servono a definire rispettivamente il punto di partenza e la lunghezza totale del segmento di audio da caricare dal file, permettendo così di estrarre solo una porzione specifica del segnale audio originale. In questo caso l'inizio è stato impostato a 1 minuto e 6 secondi, mentre la durata a 22 secondi. Come primo passo si campiona il segnale nell'intervallo di tempo dato usando la funzione *librosa.load*, che restituisce un *numpy array* contenente i campioni audio denominato y , e la frequenza di campionamento sr estratta dalla traccia audio. Si precisa infatti che il parametro " $sr=None$ " serve proprio per mantenere la frequenza di campionamento originale. Si controlla poi che l'*array* y abbia dimensione 1, ovvero che il segnale sia mono, e che la relazione fra la durata della sotto-finestra, la sua frequenza di campionamento e il numero di campioni (ottenuto da " $y.shape[0]$ ") sia corretta. Nel caso in cui uno di questi due controlli fallisca, il programma termina. A questo punto, la funzione *librosa.display.waveshow* stampa la forma d'onda nel tempo del segnale audio campionato, per assicurarsi che sia stato letto correttamente. Successivamente, viene calcolato il numero di campioni per ciascuna sotto-finestra e il numero di sequenze complete in cui il segnale può essere suddiviso. Viene quindi sovrascritto y con una sua versione troncata contenente solo i campioni che verranno utilizzati per l'analisi con DFT e FFT; eventuali campioni in eccesso non vengono inclusi. Infine, mediante la funzione *reshape*, l'*array* y viene rimodellato su una matrice

bidimensionale in cui ogni riga contiene i campioni di segnale di una sotto-finestra per un totale di $n_sequenze$ righe. Pertanto, il numero di colonne della matrice è pari al numero di campioni di segnale di ciascuna sotto-finestra, indicato dalla variabile *campioni_per_sezione*. La funzione *apri_file* ritorna una tupla contenente la matrice delle sequenze dell'audio e la frequenza di campionamento.

Ritornando nel main, viene inizializzata una lista di tuple, indicata con *finestre_processate*, dove ogni tupla contiene quattro *array*: uno rappresenta la FFT, un altro la DFT e gli altri due i rispettivi spettri delle due trasformate. Si entra poi in un ciclo *for*, dove ogni riga (ovvero ogni sotto-finestra) della matrice *y* viene mandata in input alla funzione *processa_sottofinestra*; i quattro *array* ritornati dalla funzione vengono salvati in una tupla che viene aggiunta a *finestre_processate*.

La funzione *processa_sottofinestra* calcola la FFT e la DFT (la cui implementazione viene discussa nel paragrafo successivo), a cui viene applicata la funzione *np.fft.fftshift*, che ha il compito di spostare l'ordine delle componenti dello spettro per una migliore visualizzazione nel dominio della frequenza. Vengono inoltre calcolati i rispettivi spettri delle trasformate. Questa funzione ritorna quindi quattro numpy array.

```
def processa_sottofinestra (x:np.ndarray) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:  
    fft = np.fft.fftshift(np.fft.fft(x))  
    dft = np.fft.fftshift(dft_opt(x))  
    spettro_dft = np.abs(dft)**2  
    spettro_fft = np.abs(fft)**2  
    return dft, fft, spettro_dft, spettro_fft
```

La funzione *dft_opt* riceve in input un *numpy array* contenente i campioni nel dominio del tempo del segnale da trasformare. La funzione salva in *N* il numero di campioni da analizzare e crea un *array* complesso *X* di lunghezza *N* inizializzato a zero. Si esegue quindi il calcolo della DFT mediante due cicli *for* annidati: il ciclo esterno itera su ogni frequenza discreta *k*, mentre quello interno itera su *n*, che scorre attraverso tutti i campioni del segnale nel dominio del tempo da 0 a *N*-1. All'interno dei cicli annidati si calcola la DFT, che somma il contributo di ciascun campione *n* per ogni data frequenza *k*. La funzione ritorna l'*array* complesso *X* che contiene i coefficienti spettrali della DFT.

Si sottolinea che nel codice è stata inclusa per completezza, malgrado non venga mai utilizzata, anche la funzione *dft_old*, che è la funzione base da cui è stata sviluppata la *dft_opt*. Le due funzioni si differenziano solo per il fatto che la *dft_opt* utilizza il decoratore *@njit* della libreria *Numba*, che adotta la compilazione *Just-In-Time*, convertendo il codice Python in codice macchina nativo durante l'esecuzione, e migliorando così le prestazioni. La *dft_old* riceve inoltre in input l'opzione *use_tqdm* per visualizzare una barra di progresso dell'esecuzione del ciclo esterno; la *dft_opt* non ha questa funzionalità, poiché l'uso della libreria *tqdm* non è compatibile con il decoratore *@njit*.

```

# versione standard
def dft_old (segnale: np.ndarray, use_tqdm:bool) -> np.ndarray:
    N = len(segnale)
    X = np.zeros (N, dtype=np.complex_)
    if use_tqdm:
        r = tqdm(range(N))
    else:
        r = range(N)
    for k in r:
        for n in range (N):
            X[k] += segnale[n] * np.exp(-2j * np.pi * n * k / N)
    return X

# versione compilata
@jit(parallel=False)
def dft_opt (segnale: np.ndarray) -> np.ndarray:
    N = len(segnale)
    X = np.zeros (N, dtype=np.complex_)
    for k in range(N):
        for n in range (N):
            X[k] += segnale[n] * np.exp(-2j * np.pi * n * k / N)
    return X

```

Nel main vengono poi visualizzati gli spettri della DFT e della FFT: per ogni intervallo di tempo, lo spettro della DFT (a sinistra) ha accanto il corrispondente spettro della FFT. In entrambi gli spettri l'asse x è quotato in kHz, mentre l'asse y in dB. Per fare ciò è stata usata la funzione **scala_sottofinestra**, che riceve in input uno spettro e ritorna due *numpy array* che rappresentano i valori da visualizzare sugli assi quotati rispettivamente in kHz per l'asse x e in dB per l'asse y. Per ogni intervallo di tempo viene inoltre salvata nella variabile *errore* la discrepanza fra i due spettri, calcolata come il MSE (*Mean Squared Error*) tra lo spettro della DFT e quello della FFT. Nel titolo del grafico di ogni DFT viene stampato il valore della discrepanza rispetto alla FFT calcolato come sopra indicato.

```

def scala_sottofinestra (spettro:np.ndarray, sr: int | float) -> tuple[np.ndarray, np.ndarray]:
    frequencies = np.fft.fftfreq(spettro.shape[0], d=1/sr)
    frequencies_shifted = np.fft.fftshift(frequencies)
    frequencies_kHz = frequencies_shifted / 1000
    return frequencies_kHz, 10*np.log10(spettro)

```

Segue l'analoga elaborazione del brano rock.

Infine, si accostano i grafici della FFT del brano classico e di quella del brano rock.

Prima di andare a commentare i risultati ottenuti dall'analisi dei segnali, si evidenzia il tempo richiesto per l'esecuzione della funzione *dft_opt*: quando $M=0.25$ il tempo richiesto è circa 10 minuti, quando $M=0.5$ poco meno di 19 minuti e quando $M=1$ circa 40 minuti.

M=0.25

M=0.5

 $M=1$

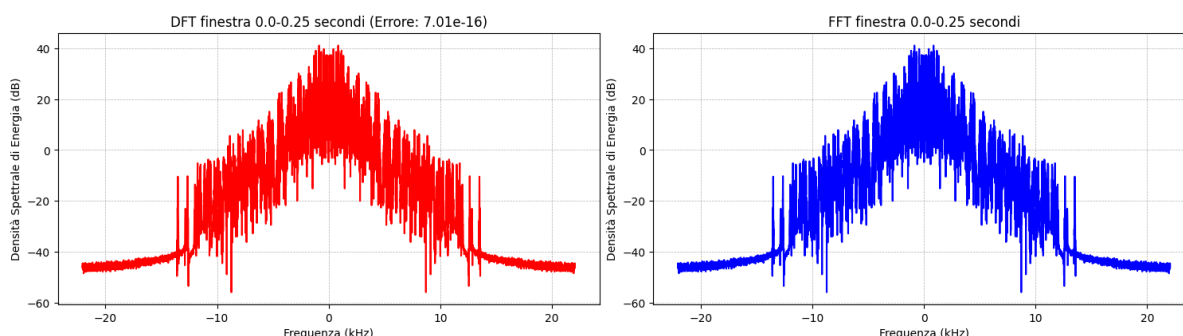
In primo luogo si può notare che in entrambi i brani, indipendentemente dai valori di M , gli spettri corrispondenti a finestre temporali che distano fra di loro 10 secondi presentano un contenuto spettrale differente. Questo è dovuto al fatto che i campioni nel dominio del tempo contenuti in ciascuna finestra rappresentano frammenti diversi del brano caratterizzati da un diverso contenuto frequenziale.

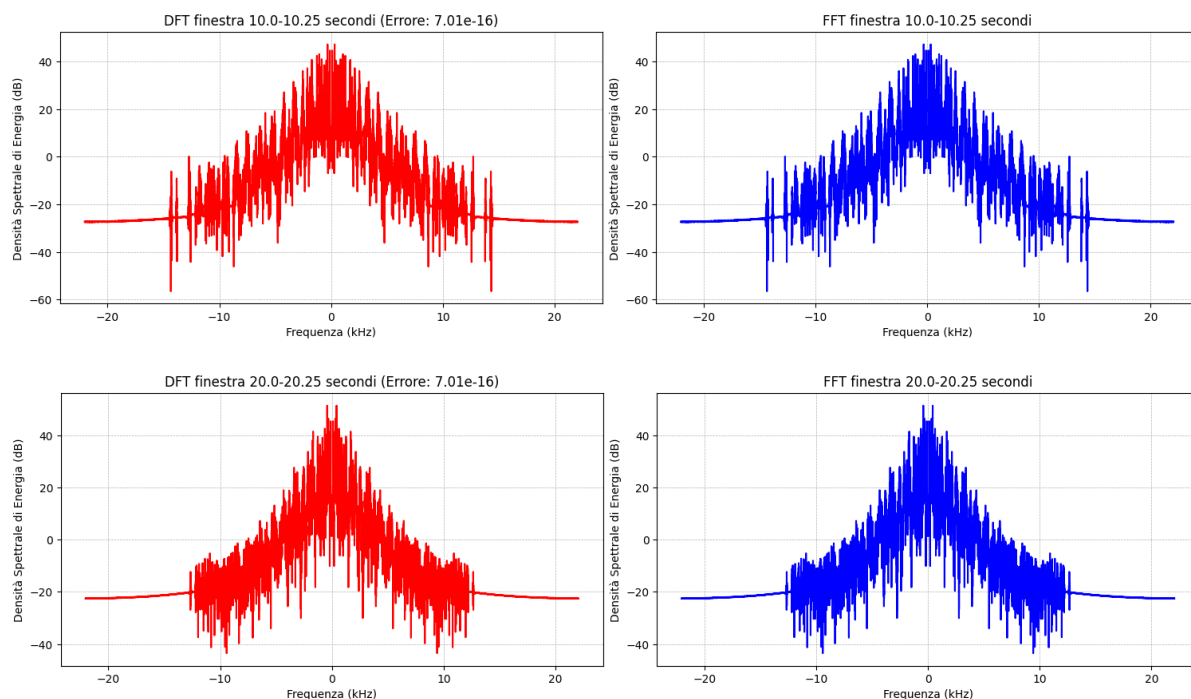
In secondo luogo, in entrambi i brani, al crescere del valore di M , lo spettro risulta essere più continuo, ovvero ci sono più frequenze rappresentate e meno “buchi” nello spettro. Questo potrebbe essere dovuto al fatto che una finestra temporale più estesa contiene un maggior numero di campioni del segnale nel dominio del tempo e quindi una maggiore varietà di frequenze nel dominio della frequenza.

Inoltre, confrontando fra loro gli spettri dei due diversi tipi di brani (musica classica rappresentata in rosa e musica rock in verde) ottenuti tramite la FFT si nota che la musica rock ha un contenuto frequenziale più esteso rispetto a quello della musica classica. Intuitivamente la musica rock ha delle variazioni temporali più rapide e dinamiche rispetto a quelle della musica classica, che si possono ottenere solamente in presenza di un componenti a frequenze più elevate.

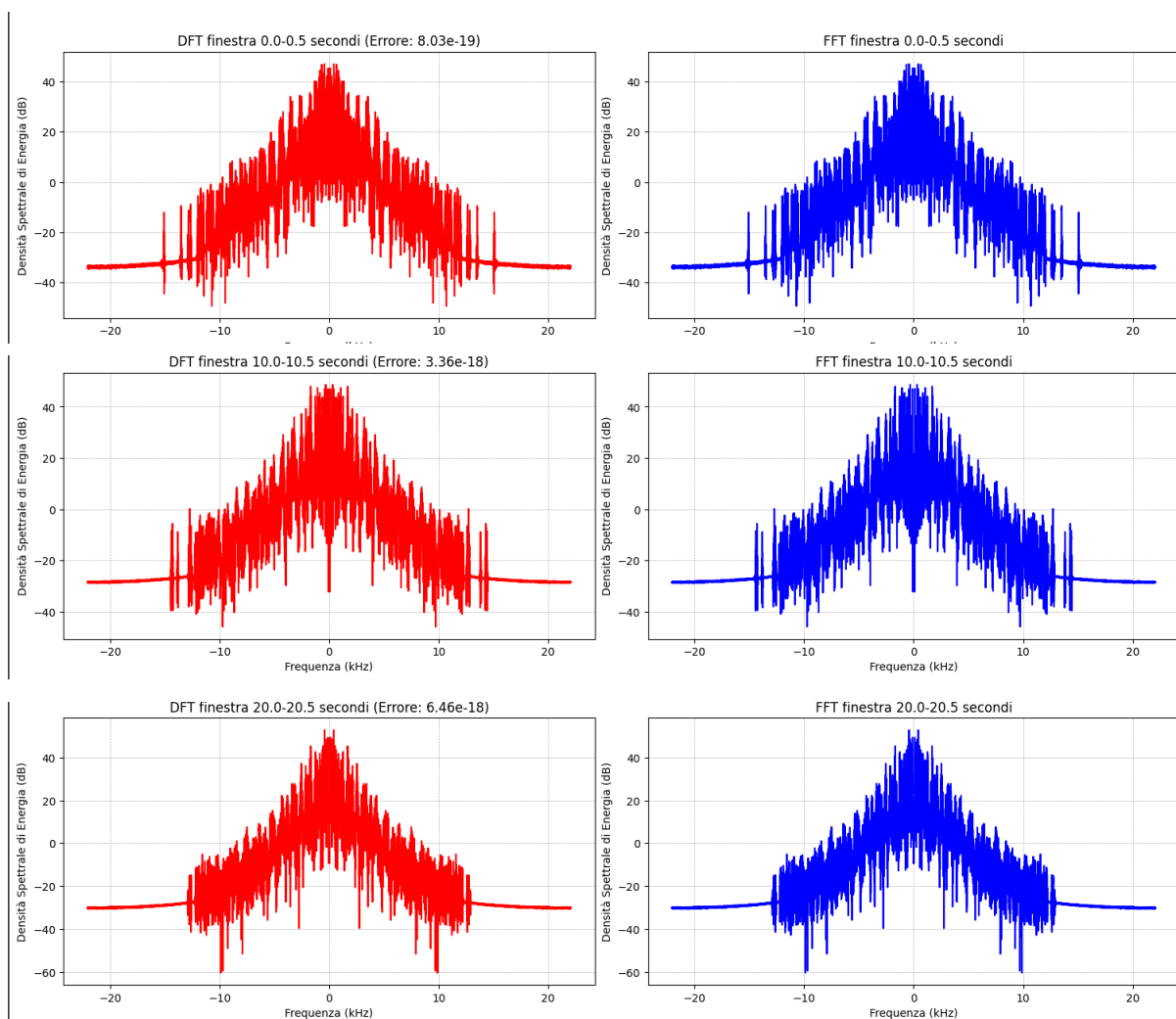
Infine, si osserva che lo spettro della musica rock ha un contenuto energetico (valori in decibel sul grafico) maggiore rispetto a quello della musica classica. Tuttavia questo potrebbe essere dovuto semplicemente al fatto che il volume del segnale corrispondente alla musica rock è più elevato rispetto al volume del segnale della musica classica.

Alcuni estratti dello spettro del brano di musica classica campionato con $M=0.25$

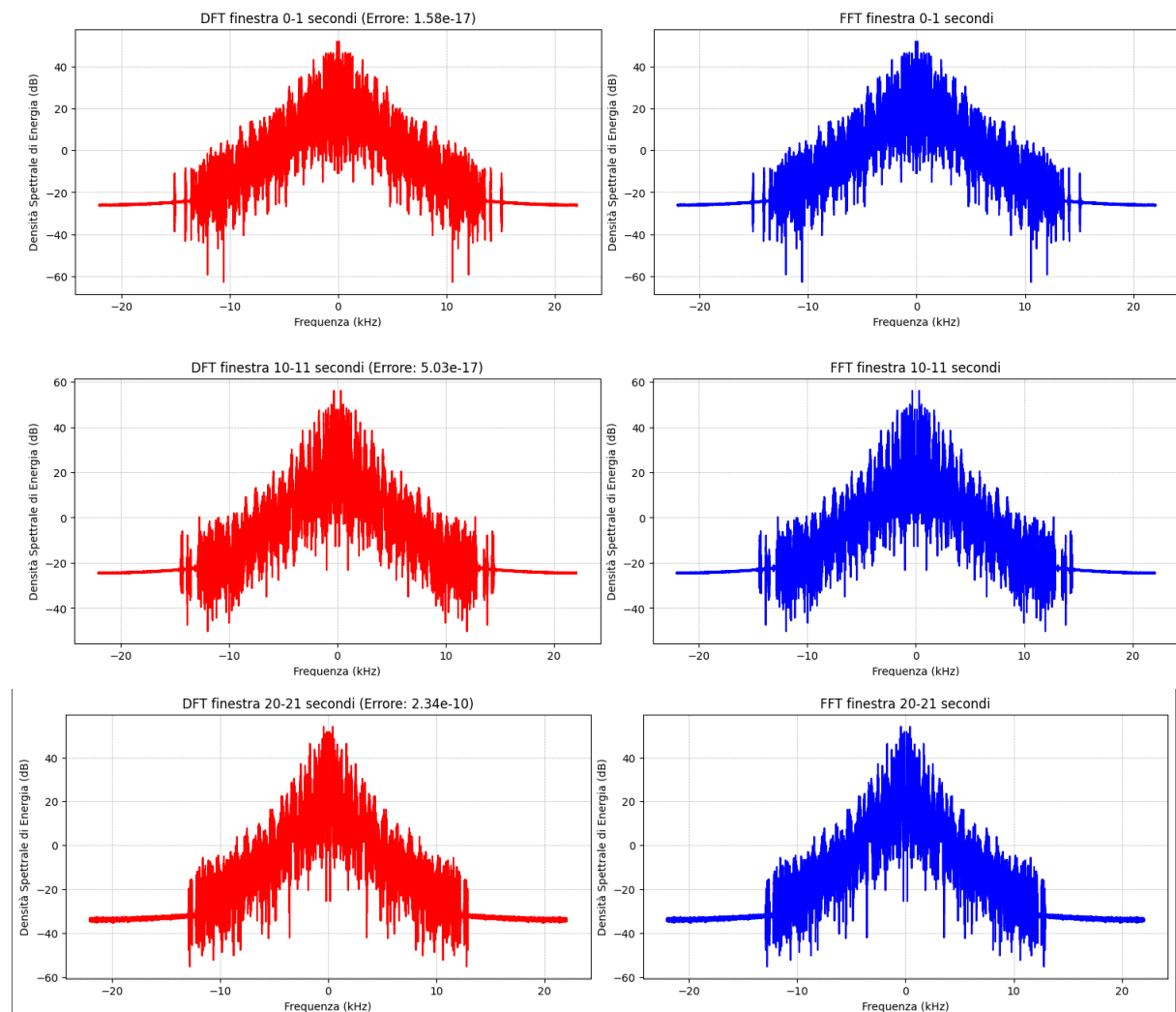




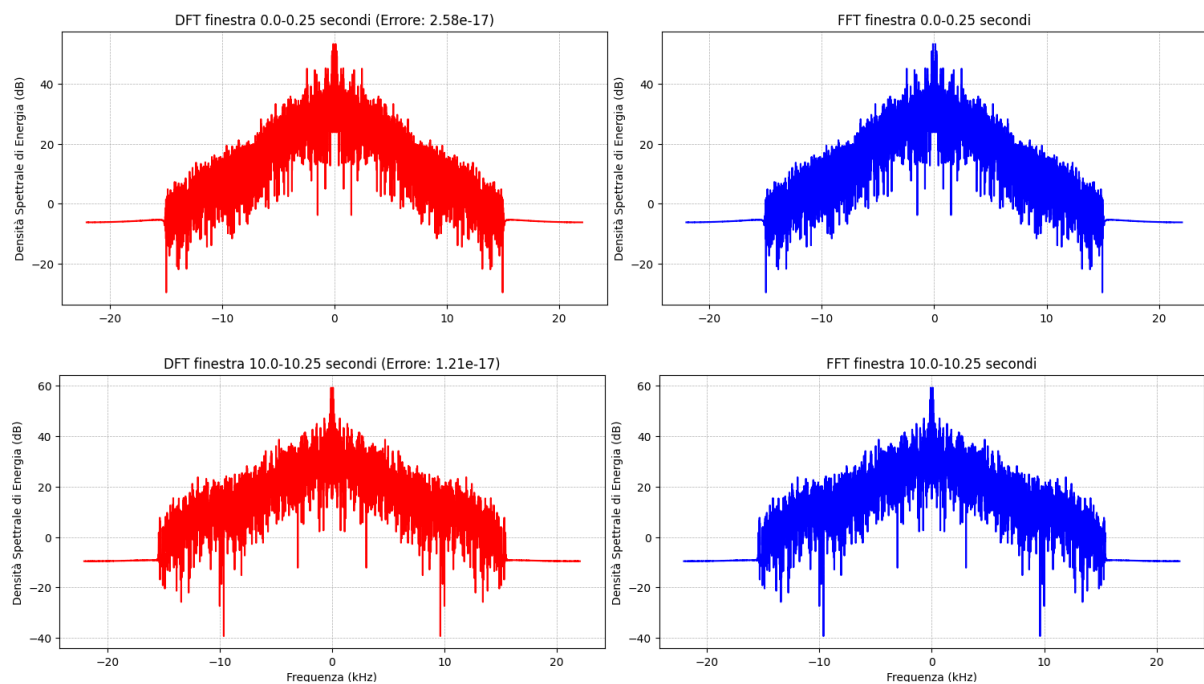
Alcuni estratti dello spettro del brano di musica classica campionato con $M=0.5$

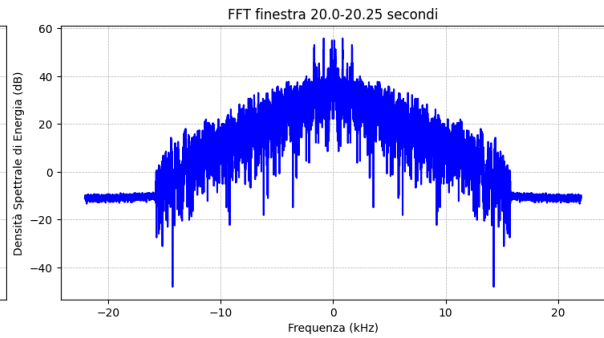
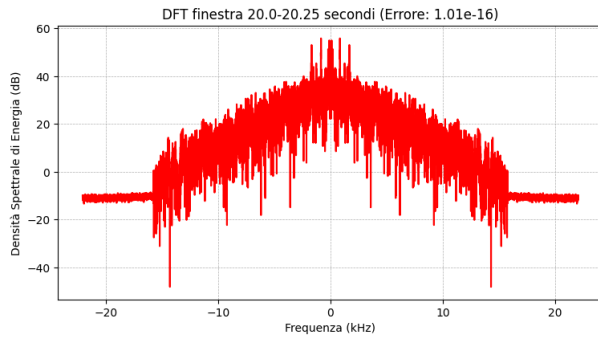


Alcuni estratti dallo spettro del brano di musica classica con $M=1$

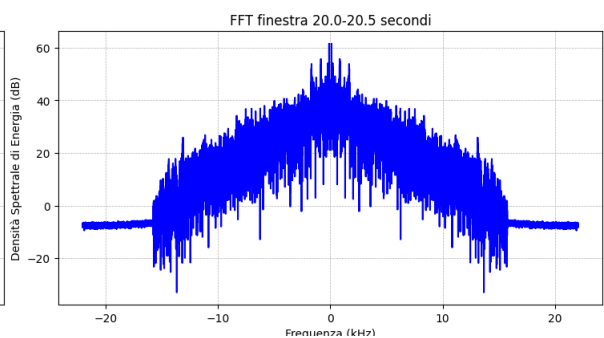
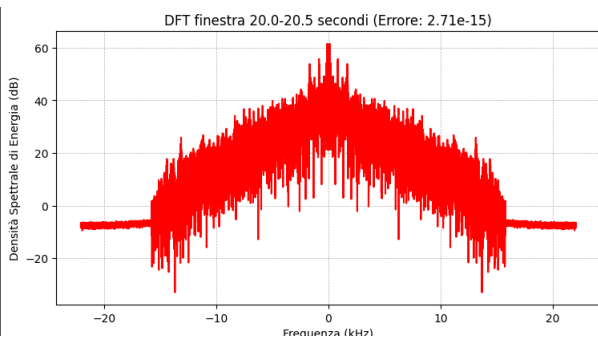
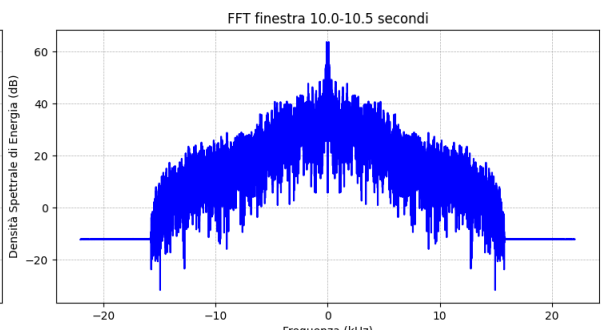
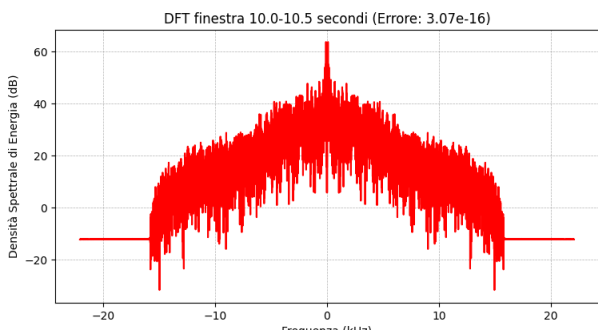
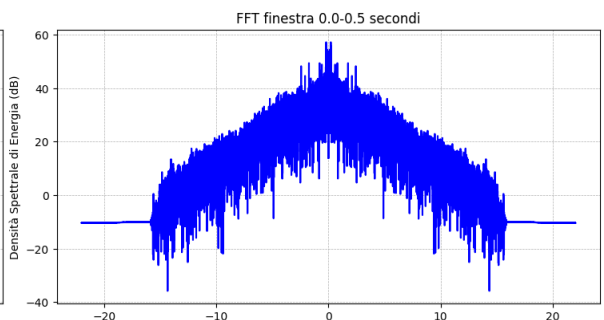
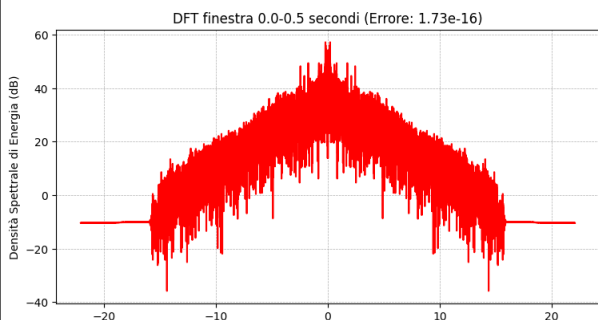


Alcuni estratti dallo spettro del brano di musica rock con $M=0.25$

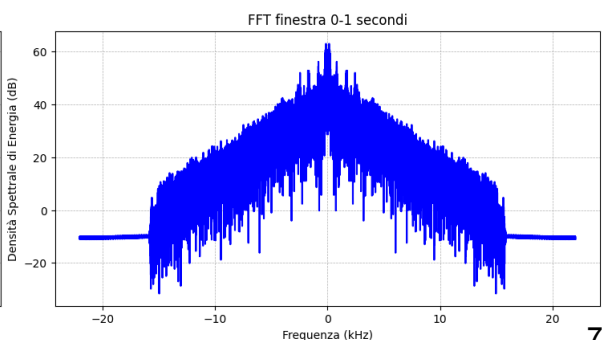
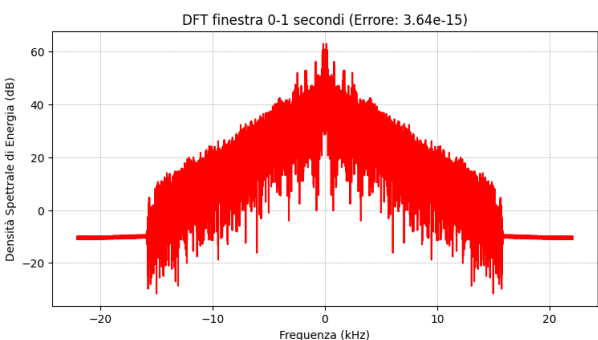


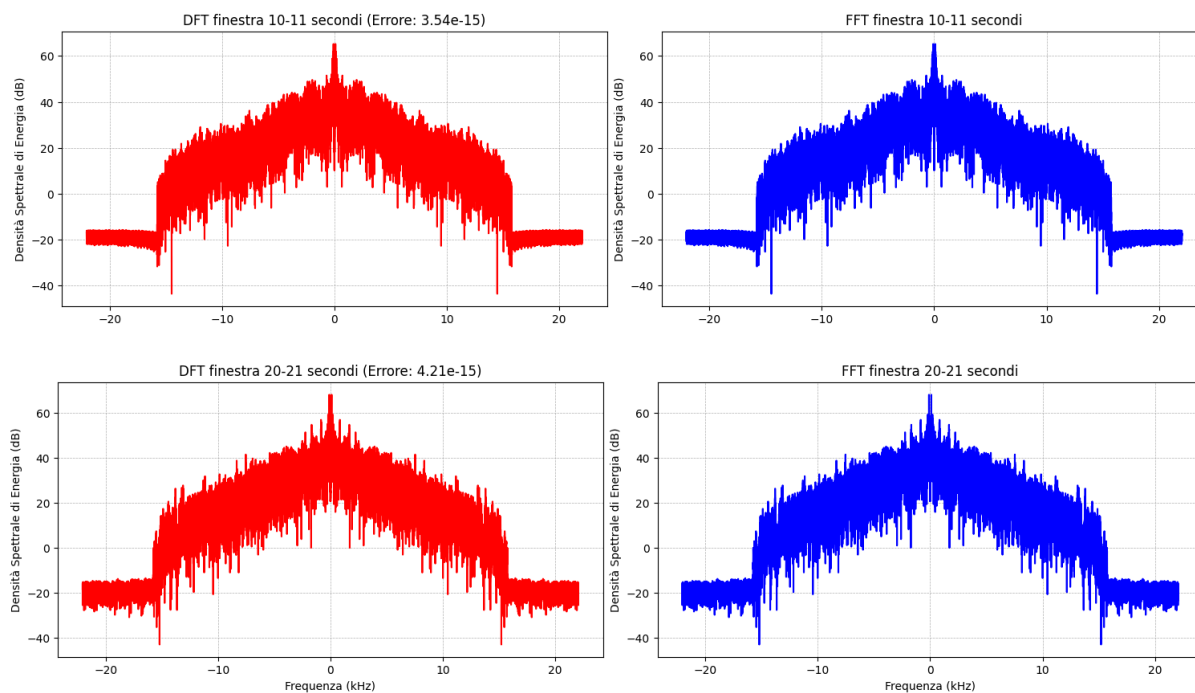


Alcuni estratti dallo spettro del brano di musica rock con $M=0.5$



Alcuni estratti dallo spettro del brano di musica rock con $M=1$





Alcuni estratti dagli spettri dei due brani sovrapposti ottenuti tramite la FFT con $M=0.5$. Il brano di musica rock è rappresentato in verde, mentre quello di musica classica in rosa.

